

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Вискворкин А. В.

Преподаватель: Бахарев В. Д.

Оценка: _____

Дата: 30.09.26

Москва, 2026

Постановка задачи

Вариант 7.

Родительский процесс создаёт дочерний процесс. Первой строчкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла с таким именем на чтение. Стандартный поток ввода дочернего процесса переопределяется открытым файлом. Дочерний процесс читает команды из стандартного потока ввода. Стандартный поток вывода дочернего процесса перенаправляется в `pipe1`. Родительский процесс читает из `pipe1` и прочитанное выводит в свой стандартный поток вывода. Родительский и дочерний процесс должны быть представлены разными программами.

В файле записаны команды вида: «число число число<newline>». Дочерний процесс считает их сумму и выводит результат в стандартный поток вывода. Числа имеют тип `float`. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)`; – создаёт дочерний процесс.
- `int pipe(int *fd)`; – создаёт однонаправленный канал для связи между процессами.
- `ssize_t write(int fd, const void* buf, size_t count)`; – записывает `count` байт из `buf` в файл с дескриптором `fd`.
- `ssize_t read(int fd, void* buf, size_t count)`; – читает `count` байт из файла с дескриптором `fd` в `buf`.
- `int open(const char* pathname, int flags, mode_t mode)`; – открывает файл, возвращает файловый дескриптор.
- `int close(int fd)`; – закрывает файловый дескриптор.
- `int dup2(int oldfd, int newfd)`; – дублирует файловый дескриптор `oldfd` в `newfd`.
- `int execv(const char* path, char* const argv[])`; – заменяет текущий образ процесса новым по пути `path`, передавая в него аргументы `argv`.
- `pid_t wait(int* wstatus)`; – ожидание завершения дочернего процесса.
- `void exit(int status)`; – завершает процесс с кодом возврата `status`.

Parent принимает из стандартного потока ввода имя файла, введённое пользователем, после чего открывает этот файл в режиме чтения. Затем parent создаёт однонаправленный поток `child_to_parent` для взаимодействия с созданным дочерним процессом `child`. Стандартный поток ввода `child` переопределяется файлом, открытым родительским процессом, а стандартный поток вывода – потоком `child_to_parent`. Child считывает данные из файла с помощью `read`, преобразует прочитанные данные в числа типа `float` и вычисляет сумму чисел на одной строке, после чего выводит сумму в свой стандартный поток вывода. Parent читает полученное от дочернего процесса и выводит в свой стандартный поток вывода. Суммы выводятся с точностью 6 знаков после запятой.

Код программы

parent.c

```
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <fcntl.h>
```

```

static char CHILD_PROGRAM_NAME[] = "child";

int main(int argc, char **argv) {
    char filename[1024];

    const char msg[] = "Enter filename: ";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);

    ssize_t len = read(STDIN_FILENO, filename, sizeof(filename) - 1);
    if (len < 0) {
        const char msg[] = "error: failed to read filename\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
    filename[len] = '\0';
    if (len > 0 && filename[len - 1] == '\n')
        filename[len - 1] = '\0';

    int file = open(filename, O_RDONLY);
    if (file == -1) {
        const char msg[] = "error: failed to open file\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    int child_to_parent[2];
    if (pipe(child_to_parent) == -1) {
        const char msg[] = "error: failed to create pipe\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    const pid_t child = fork();

    switch (child) {
    case -1: {
        const char msg[] = "error: failed to spawn new process\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    } break;

    case 0: {
        dup2(file, STDIN_FILENO);
        dup2(child_to_parent[1], STDOUT_FILENO);
        close(file);
        close(child_to_parent[0]);
        close(child_to_parent[1]);
        {
            char *const args[] = {CHILD_PROGRAM_NAME, NULL};
            int status = execv(CHILD_PROGRAM_NAME, args);
            if (status == -1) {
                const char msg[] = "error: failed to exec\n";
                write(STDERR_FILENO, msg, sizeof(msg) - 1);
                exit(EXIT_FAILURE);
            }
        }
    } break;

    default: {
        close(file);
        close(child_to_parent[1]);

        char buf[4096];
        ssize_t bytes;
    }
    }
}

```

```

        while (bytes = read(child_to_parent[0], buf, sizeof(buf))) {
            if (bytes < 0) {
                const char msg[] = "error: failed to read from pipe\n";
                write(STDERR_FILENO, msg, sizeof(msg) - 1);
                exit(EXIT_FAILURE);
            }

            write(STDOUT_FILENO, buf, bytes);
        }

        close(child_to_parent[0]);
        wait(NULL);
    } break;
}

return 0;
}

```

child.c

```

#include <unistd.h>
#include <stdlib.h>
#include <string.h>

void stdout_float(float val) {
    char buf[64];
    size_t i = 0;

    if (val < 0) {
        buf[i++] = '-';
        val = -val;
    }

    long long left = (long long)val;

    char temp[64];
    size_t j = 0;
    if (left == 0)
        temp[j++] = '0';
    while (left > 0) {
        temp[j++] = (char)('0' + left % 10);
        left /= 10;
    }
    while (j > 0)
        buf[i++] = temp[--j];

    buf[i++] = '.';
    float frac = val - (long long)val;
    for (int k = 0; k < 6; k++) {
        frac *= 10.0;
        int d = (int)frac;
        buf[i++] = (char)('0' + d);
        frac -= d;
    }

    buf[i++] = '\n';
    write(STDOUT_FILENO, buf, i);
}

```

```

void print_sum(char* line, size_t len) {
    line[len] = '\0';

    char* p = line;
    float sum = 0.0;

    while(*p != '\0') {
        char *end;
        float val = strtod(p, &end);

        if (end == p)
            p++;
        else {
            sum += val;
            p = end;
        }
    }
    stdout_float(sum);
}

int main(int argc, char **argv) {
    char buf[4096];
    size_t i = 0;
    ssize_t bytes;

    while (1) {
        bytes = read(STDIN_FILENO, buf + i, sizeof(buf) - 1 - i);
        if (bytes < 0) {
            const char msg[] = "error: failed to read from stdin\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
        }
        if (bytes == 0) {
            if (i > 0)
                print_sum(buf, i);
            break;
        }

        i += (size_t)bytes;
        size_t start = 0;
        for (size_t j = 0; j < i; j++) {
            if (buf[j] == '\n') {
                print_sum(buf + start, j - start);
                start = j + 1;
            }
        }
        if (start > 0) {
            size_t unfin = i - start;
            if (unfin > 0)
                memmove(buf, buf + start, unfin);
            i = unfin;
        }

        if (i >= sizeof(buf) - 1) {
            print_sum(buf, i);
            i = 0;
        }
    }
}

```

```
    }  
}  
  
    return 0;  
}
```

Протокол работы программы

TestOne

1.23 4.48 25.17

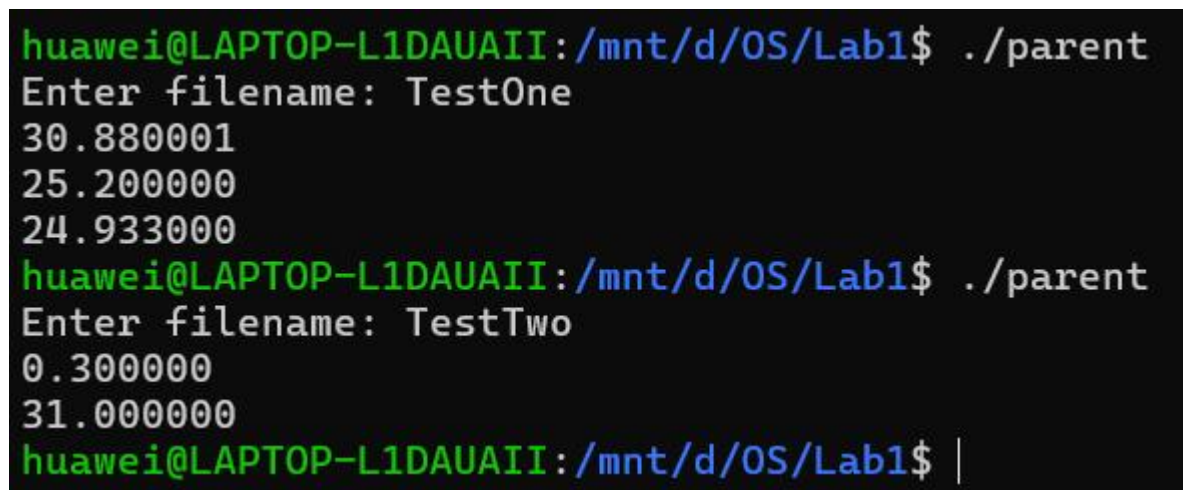
15.2 10

3.333 9 12.6

TestTwo

0.1 0.2

15 2 2 4 8



```
huawei@LAPTOP-L1DAUAI: /mnt/d/OS/Lab1$ ./parent  
Enter filename: TestOne  
30.880001  
25.200000  
24.933000  
huawei@LAPTOP-L1DAUAI: /mnt/d/OS/Lab1$ ./parent  
Enter filename: TestTwo  
0.300000  
31.000000  
huawei@LAPTOP-L1DAUAI: /mnt/d/OS/Lab1$ |
```

Вывод

В результате выполнения лабораторной работы были улучшены навыки работы с процессами и межпроцессным взаимодействием в UNIX-подобных системах. Освоены навыки работы с неименованными каналами, созданием дочерних процессов, перенаправлением потоков ввода-вывода. Получены навыки работы с низкоуровневым вводом-выводом с помощью системных вызовов.